Islands of Reach: A Censorship-Resistant Federated Forum for National Internet Blackouts

Anonymous submission

***Abstract*—An Internet shutdown is rarely a clean power switch. Transit to the outside world can be withdrawn while domestic networks keep carrying packets among themselves, leaving reachable fragments that recent measurement work calls connectivity islands. We ask a narrow question: when some domestic IP paths survive, can independently operated forum servers keep accepting signed posts and reconciling them across whatever graph remains? Our prototype composes four familiar mechanisms for this regime — canonical protobuf envelopes admitted as the exact bytes their author signed, operator receipts bound to those bytes, caller-initiated synchronisation that survives asymmetric reachability, and an explicitly trusted dual-homed relay between islands. On a four-node testbed whose shared exchange segment is withdrawn mid-run, a dependent certificate, community and post crossed the severed boundary in 0.5, 1.2 and 2.1 seconds, within a factor of two of the same chain measured while the exchange was intact. An eight-node chain delivered 200 of 200 envelopes seven hops with a 4.125 s median. TypeScript, Rust and Python implementations agreed on all 16 canonical vectors, and a signature-invalid flood produced no database writes. Against a median gzipped ActivityPub activity our envelopes are 4.4–6.9× smaller, at the cost of Fediverse interoperability and JSON-LD extensibility. These results establish partition tolerance under surviving IP paths on a single-host testbed, and nothing about total blackouts, real inter-provider deployment, or resourced denial of service.**

***Index Terms*—federated systems, Internet shutdowns, network partitions, signed objects, store-and-forward**

I. Introduction

Internet shutdown doesn't necessarily mean a complete loss of connectivity. The disruption can occur in several forms like reduction in bandwidth, blocking destinations, filtering protocols, and loss of international routes. Despite these restrictions, domestic networks can still communicate with each other [1], [2]. Baltra et al. described the groups that lost the core internet connection but still can communicate locally as "islands" [3]. They also described networks that still have limited connection with the internet as "peninsulas" [3]. They have explained how these disrupted situations can be identified and developed measurements for them. Our work focuses on developing a system which can continue working in these crucial situations. Our system tries to answer a specific question: if the international routes are cut but domestic IP connection remains connected and a forum server works independently, whether they can receive signed posts and synchronize those posts across the network?

Other existing systems focus on a related problem which is not exactly similar to ours. Circumvention systems are designed to reach a blocked region. Snowflake, for instance, routes a blocked user out through temporary proxy servers [4]. Such systems must reach either cooperating network infrastructure or a proxy, and when international transit is withdrawn they can reach neither. Anix, Rangzen and Moby instead construct a device-to-device or mobile mesh network for situations where general internet access is unavailable [5]–[7]. So far we've studied, none of them have examined whether independent servers can continue to check, store and relay messages over a surviving local network. Our research addresses this gap.

This paper makes mainly 4 contributions:

1) Server follows same byte preserving admission process for both locally saved object and object found from other servers. So, a protocol-compliant relay does not change a malformed object into a valid one.
2) It supports caller-initiated delivery, streaming and backfill. So, even if a server can't accept an incoming connection, can still publish and recover objects by establishing an outbound connection to known peer.
3) It defines a quota bound relay policy. Relay verifies the object again, checks quota. Most importantly, it doesn't send the object back to the server again from which it has received it. A trusted peer may get larger quota and access to additional traffic classes while sending or receiving messages. But it doesn't mean that a post by a trusted peer is automatically valid.
4) It reports a result from a testbed where connection between islands were disconnected while experiment. The experiment also includes cross-language byte agreement tests, performance change with respect to increasing hop, flood of invalid signatures and a comparison with ActivityPub.

Our claim is limited to partition tolerance only when some IP paths remain between servers directly or through a bridge. This proposed designed cannot create a new route during complete blackout. This cannot force a server to accept a valid post or hide any visible relay from its hosting network operator.

II. Background and Research Gap

A. Federated and Signed System

ActivityPub directly delivers social activities to the server's inbox [8]. AT protocol supports federation and repository replication [9]. Nostr replicates objects signed by the author [10]. They generally assume that some relevant server, relay, or peer endpoint remains reachable. But none of them has evaluated exact-byte admission, operator receipts, restoration

through outbound connection and relay policy for connection between two network islands.

B. Communication During Blackout

Anix, Rangzen and Moby establish a network connecting nearby devices [5]–[7]. These systems work without internal server connection. But they sacrifice bandwidth, devices extra power consumption and network capacity. Our system chooses the opposite trade-off: when the servers are connected, it establishes connection between them, but can't work when all IP paths are totally disconnected. Those systems don't compete with one another, rather they complement each other. So, this paper doesn't compare their latency, because they are designed for different workloads.

C. Accountability:

Append-only logs, signed tree heads and inclusion proofs were introduced by Certificate Transparency to make certain classes of misbehaviour auditable after the fact [11]. PeerReview and CoSi explores several stronger forms of accountability by using witnesses and cosigning [12], [13]. Our receipt has been kept comparatively weaker intentionally. It only proves that an operator has acknowledged a specific content ID at a stated time. It doesn't prove that operator will receive valid object, will forward object faster or object will not be rejected silently.

D. Study gap and design objective

There is a difference currently present two fields. Normally routed federation assumes that server is always directly connected to the network. On the other hand, device to device assumes that network is totally disrupted. We've studied an intermediate situation between them where independent servers are working and there exists internal paths while the international routes are completely disconnected. In this situation, our goal is to admit each signed object once per server, keep its signed bytes unchanged, and allow every reachable server using the same rules and prerequisite state to reach the same validity decision.

III. System Design

A. Architecture

The system is built on two premises: a forum should keep working as connectivity narrows, and a server's acceptance of content should remain externally auditable. We evaluate two degrees of narrowing — a national partition, where international transit is gone but the domestic exchange still carries traffic, and an ISP island, where inter-ISP peering has failed and a dual-homed relay is the only route between two islands. Clients reach nodes over HTTP and nodes federate over gRPC; every claim in this paper is scoped to that IP path.

The canonical signed envelope is invariant across ingress media and relay paths. The transport context and admission context are not. Every node validates the envelope through the same 19-stage admission pipeline. The first 12 stages validate the envelope without persistent writes. If the envelope is accepted, the node commits it, signs a receipt, and places a federation task in a durable outbox. The scheduler drains the outbox by priority class, then by '(destination, source uplink, traffic class)'.

The node must still discriminate how an envelope arrived. The same envelope may be submitted by a client or relayed by a peer; its bytes, content identifier and author signature are identical either way. What differs is the admission context — ingress kind, supplying peer, direction, source uplink, traffic class and quota policy — which governs deduplication, replay protection, scheduling and fan-out. It never relaxes signature, identifier, or body validation.

A bridge is an ordinary node running the same stack, dual-homed onto two islands; but not a transparent packet forwarder. Its relaying capability has already been independently validated, stored, and quota-bounded under its own key.

Independent audit-log services (ALSs) provide an external proof of server acknowledgement. This holds the server owners accountable. They operate outside the write path. After a server commits the write it returns a signed receipt. The client builds a public audit certificate, stores it locally, and then forwards it to one or more ALSs. Each ALS independently re-verifies the canonical envelope, content identifier, receipt, signed tree head, and inclusion proof before appending the certificate to its own hash-chained record. If any server later hides or deletes the accepted object, the acknowledgement remains externally verifiable. This supports censorship resistance by limiting a single server's ability to silently erase evidence of acceptance.

B. Signed objects and admission

An envelope carries a domain tag, an object type, a payload, a public key, a nonce, and an expiry. Its identifier is the SHA-256 digest of a specified canonical byte string, and an Ed25519 signature covers that same string [14]. Protobuf does not guarantee a unique serialisation [15], so we constrain a profile on top of it — ascending field order, minimal varints, no unknown fields retained — and add the rule that actually matters: decoding and re-encoding an object must reproduce the bytes as received, exactly. An object that fails that comparison is rejected outright, never repaired.

Verifying the exact received bytes after a strict decode is established practice in DER, JWS, and Certificate Transparency leaf formats. In this system, the rule protects federation. If a relay decoded a peer's bytes and re-encoded them before checking the signature, it could normalise a non-canonical submission into the canonical form that was signed. The relay would then verify reconstructed bytes rather than the bytes that arrived. Our decode–re-encode comparison prevents a receiver from repairing or normalising attacker-controlled bytes before verification. As a result, no relay can make a non-canonical or invalid submission acceptable by rewriting it.

Fig. 1 lays out the admission path. Stages 1 through 12 — size, parsing, canonicality, domain separation, identifier derivation, signature, expiry, replay, authorisation — write

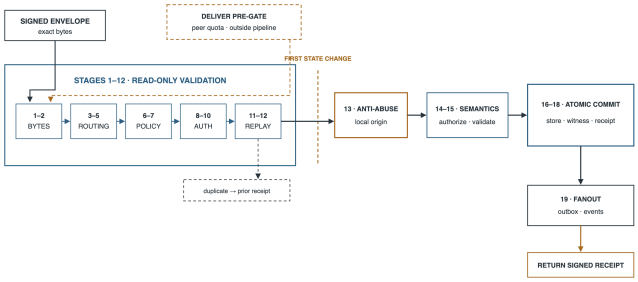

Fig. 1. Admission pipeline. State changes begin only after the cheap byte and validity checks have all succeeded.

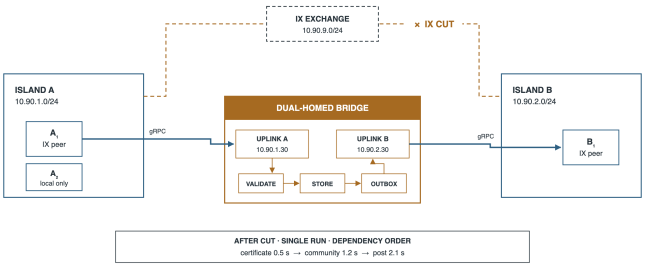

Fig. 2. Route-cut topology. Four application nodes span two isolated network segments joined by a shared exchange, which the experiment withdraws mid-run.

nothing to persistent storage, so an invalid flood cannot amplify into write load, a property we measure directly in §V. The stages after that charge origin pricing or a receiver quota, commit the raw object with its derived indexes in one transaction, append its hash to a Merkle log, and only then enqueue federation. Exact retries are made idempotent under concurrency by a uniqueness constraint in the database, not by a read-then-write check that a race could defeat.

On acceptance, the server signs a receipt binding the content identifier, the leaf position, the resulting signed tree head, and an inclusion proof. The acknowledgement therefore commits to the exact bytes admitted. Peers also exchange signed tree heads containing a tree size, root hash, and timestamp. Two heads of equal size with different roots are an unambiguous conflict. A larger head is recorded as newer, but it is treated as proven only after consistency proof is obtained. A smaller head is also not immediately treated as a fork, as observations may arrive out of order: in case the timestamp is older than the recorded head, it is discarded as stale; otherwise, a smaller size is reported as a possible fork. This is deliberate stale-head handling. It removes a class of self-inflicted false positives and catches outright tree shrinkage, however it is not full fork detection: A adversary who rewrites history while keeping the tree size non-decreasing can pass these local checks, and equivocation between two peers cannot be detected by a single local comparison. Detecting those cases requires consistency proofs and independent cross-observer exchange, which the network partitions we target are likely to obstruct.

C. Partition-aware federation

Three caller-initiated RPCs cover ordinary delivery and recovery. DELIVER pushes a batch of accepted envelopes to a peer. STREAMACTIVITIES resumes a live stream from a durable cursor. BACKFILL requests objects that the node knows it missed, using a resumable log position. Because all three operations are initiated by the caller, a node does not need to accept inbound federation connections. A server behind carrier-grade NAT can therefore federate in both directions over connections it opens itself. This solves reachability asymmetry, not discovery: at least one peer address must already be known.

The scheduler keeps an independent durable queue for each destination peer. It drains these queues concurrently under bounded leases, with deadlines, exponential backoff, and idempotency keys. This structure exists because its absence was a real defect, described in §V: a serial drain allowed one unreachable peer to block delivery to every other peer, including the bridge. One caveat belongs here rather than in the limitations section. A delivery deadline bounds how long the drain pass waits on a peer; it does not cancel the underlying transport call. Because the sender holds no cancellation token, a stuck call can continue to hold resources after its deadline, even though it no longer blocks delivery to other peers.

Fig. 2 shows the route-cut topology used in the evaluation. Island A contains two ordinary nodes, island B contains one, and a shared exchange segment stands in for an Internet exchange point. The bridge holds one address on each island network. Once the exchange segment is withdrawn, both islands must already have configured trust in the bridge. That trust determines the rate and priority of the bridge's traffic; it never determines whether an envelope is valid. The bridge validates and stores every object it relays, enforces quota per uplink pair and traffic class, and never sends an object back to the peer from which it arrived.

Four implementation rules carry most of the replication safety. Peer bytes are passed through unchanged: the transport passes received envelopes as opaque byte strings and does not decode or re-encode them. Trust controls quota and priority, never validity; every peer, including the bridge, still repeats the full admission checks. Duplicate delivery is inevitable when a live stream and a backfill race, and it is absorbed by a database uniqueness constraint rather than by a read-then-write check. Fan-out excludes the peer that supplied an object, avoiding a redundant round trip that deduplication would eventually terminate, but not for free.

IV. METHODOLOGY

We evaluate five questions: whether independent encoders agree on the canonical form, whether objects cross an enforced route cut, how delivery latency scales with hop count, whether malformed requests produce storage writes, and how the representation compares in size with ActivityPub. All experiments

run on one physical host; §VI treats the consequences of that choice at length.

Correctness tests. A shared corpus of 16 positive and adversarial byte vectors is executed independently by the TypeScript, Rust, and Python implementations. Each must agree byte-for-byte on encoding and identifier derivation, and each must reject every negative vector. A further 85 focused tests cover ingress (29), the outbox (3), federation (31), and simulated ISP behaviour (22). These are regression tests written by the system's authors, not an independent security audit.

Route-cut experiment. Four application nodes, two databases, and two caches are distributed across three isolated virtual network segments, as shown in Fig. 2. Containers supply reproducible layer-2 isolation and let the harness withdraw the only configured inter-island path mid-run; nothing in the design depends on that packaging.

The test runs in four stages. It first reads kernel TCP state from `/proc/net/tcp` inside the bridge to confirm the local source address each established federation connection left from. It then withdraws the shared exchange segment and verifies that direct inter-island probes fail, and that island B does not already hold the content to be submitted. Finally it publishes a dependency chain — an author certificate, a community record, and a post requiring both — and polls island B for each exact content identifier. The chain is ordered because a post cannot be rendered without its certificate and community, so the crossing is measured across all three objects rather than the post alone. Nineteen assertions cover topology, source binding, relay behaviour, quota reservation and uplink failover. Reported latencies include the application's polling interval and are therefore upper bounds.

Hop-scaling chain. Eight nodes are peered into a line, each with its own database, so that a node i hops from the origin is reachable by exactly one path of exactly that length. Hop count is the independent variable. A mesh would destroy this property, since in a mesh every node is one hop away and only fan-out width varies.

Latency is computed by subtracting the `received_at_ms` value each node writes inside the same transaction as its projection. No polling observer sits in the measurement path. This is sound only because all nodes share the host clock. We inject 200 valid envelopes at hop 0 at a paced rate and record arrival at hop 7. The eight nodes share one database process with a database each, which adds contention and inflates the reported figures.

Signature-invalid flood. Random bytes are refused within microseconds at the parse stage and measure nothing interesting. The tool therefore lifts a genuine envelope from the node's own log and mutates a single byte inside the signed region. This invalidates the signature and changes the content identifier, so every request is novel and none can be short-circuited by deduplication. We issue 3,000 such requests at concurrency 16 and record completion rate, rejection stage, the fraction reaching signature verification, and—via the database's collection-level profiler—the resulting writes. The ordinary rate limiter stays enabled, so this is not a saturation benchmark.

Encoding comparison. We collected 80 real `Create`/`Note` activities from the public outboxes of two stock ActivityPub deployments. Because a collection hoists `@context` to the collection while an activity POSTed to a remote inbox carries its own, measuring the bare outbox item would understate delivery size. The tool therefore refetches a standalone object from the same host and reattaches exactly those bytes. We compare raw and gzipped activity bytes against equivalent signed envelopes, subtracting authored UTF-8 text so the metric reflects what a protocol designer controls rather than how much a user typed.

The comparison is deliberately generous to ActivityPub. It excludes HTTP framing, TLS, per-follower fan-out, shared dictionaries, and—most significantly—authentication, since our bytes carry a 64-byte signature verifiable at rest while HTTP Signatures are transport level and vanish with the request.

V. Results

A. Correctness and convergence

All three implementations agreed on all the 16 canonical vectors and rejected all the negative ones as specified. The additional tests also checked invalid signatures, incorrect byte formats, replay attacks, missing permissions, duplicate messages, wrong signing keys, invalid proofs, outdated tree heads, and conflicting logs. Sixteen vectors cannot cover every protobuf's edge cases. However, they show that those three independently developed implementations follow exactly the same byte-level rules. This is essential for a relay network to work reliably.

The route-cut test passes all 19 checks, including the kernel TCP check confirming that federation connections left the bridge from both configured addresses, 10.90.1.30 and 10.90.2.30 — an assertion no in-process harness can make. With direct inter-island reachability withdrawn, island B received the certificate after 0.5 s, the community after 1.2 s and the post after 2.1 s, in dependency order. The same chain with the exchange intact took 1.5 s, 1.5 s and 1.9 s, so the bridged path is comparable to the direct one, within a factor of two on every object. These are single runs, not distributions.

B. Latency scaling and admission cost

All 200 envelopes successfully traveled through the eight-node chain to hop 7. The median End-to-end latency was 4.125 s and the 99th percentile 6.115 s. The per hop results which are in Fig. 3 are more useful than the final numbers. Most of the delay comes from the outbox drain interval, not the protocol itself, which if predicted roughly then it's $drain/2$ plus a fixed term; With a 1,000 ms drain interval, each hop took about 589 ms, including roughly an 89 ms fixed protocol work. With a 250 ms interval, each hop took about 196 ms, implying 71 ms of fixed work. Since this fixed cost stayed around 71–89 ms even when the drain interval changed, it actually represents the protocol's actual per-hop cost. This

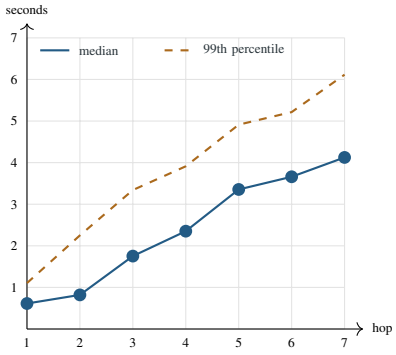

Fig. 3. Per-hop latency measured from in-transaction timestamps across the eight-node chain at a 1,000 ms drain interval. All the 200 envelopes reached hop 7.

includes verifying, projecting, appending, and re-queuing the message. The remaining delay is mainly a scheduling choice that operators can adjust based on bandwidth and battery needs. However, these results cannot really be treated as Internet performance predictions as the test used a shared database process and did not include real wide-area network delays.

The system was able to reject the invalid-signature flood at around 730 requests per second, without writing anything to the objects, indexes, logs, or outbox. We also verified that the profiler was active during the test, since it recorded six read operations. In total, 3,000 requests were sent, and about 2,700 of them were blocked by the per-peer rate limiter before reaching signature verification. So, only around 9% reached signature verification, and then the expensive rejection process, which is the canonical parsing followed by Ed25519 verification. This ran about 66 requests per seconds. The final conclusion is a bit limited but clear enough. Invalid traffic was stopped before causing any writes, with the rate limiter handling most of the attack first. This test does not necessarily show how the system would handle floods from many different peers, valid messages that should cause writes, or long-term parser and database pressure.

Table I compares two encoding formats and their costs. Both tested hosts produced the same 927-byte `@context` declaration, which implies this is a fixed cost of the standard ActivityPub format, not something specific to our deployment. The declaration alone is more than four times larger than our entire signed forum post. Compression removes much of the apparent size advantage, so we use compressed ActivityPub numbers for a fair comparison. Gzipped ActivityPub messages were 786–1,400 B with a median of 1,074 B. Our raw signed envelopes were only 155 B, 220 B, and 243 B making ActivityPub about 4.4–6.9× larger. Our envelopes do not benefit from compression at all as the 64-byte Ed25519 signature is incompressible and gzip makes all three slightly *larger*. So the honest comparison is gzipped ActivityPub against our raw bytes, not the roughly 20× ratio suggested by comparing uncompressed sizes. However, size is the only area where our format is better. It cannot connect to the existing Fediverse, requires generated codecs and unfamiliar tools, does not allow unknown fields, and therefore requires versioned schemas for future changes.

TABLE I
REPRESENTATION SIZE AND DESIGN TRADE-OFFS AGAINST ACTIVITYPUB. SAMPLE: 80 REAL CREATE/NOTE ACTIVITIES FROM TWO STOCK DEPLOYMENTS. RATIOS COMPARE GZIPPED ACTIVITYPUB AGAINST RAW SIGNED ENVELOPES, WHICH DO NOT COMPRESS.

Dimension	This work vs. ActivityPub
Measured bytes	Ours raw: check-in 155 B, post 220 B, broadcast 243 B (gzip enlarges all three). AP delivered JSON-LD median 4,216 B, gzipped median 1,074 B (786–1,400 B); `@context` alone a fixed 927 B. Ratio 4.4–6.9×, representation only: HTTP, TLS, fan-out, shared dictionaries and AP authentication are all excluded in AP's favour.
Interoperability	Custom protobuf domains and RPCs against AP's standard actor, object and inbox model: no Fediverse compatibility and no existing application ecosystem.
Evolution and tooling	Unknown fields rejected and versioned profiles required, against a flexible JSON-LD vocabulary that is inspectable with ordinary web tools. Costs harder rolling upgrades, more cross-language test surface, and generated codecs operators must learn.
Integrity profile	One exact byte string per object, with content ID, author signature and receipt, against authentication that depends on the deployment profile and HTTP Signatures that do not survive the request. Unambiguous byte identity, but a new security profile to maintain and audit.

VI. DISCUSSION AND LIMITATIONS

Testbed validity. All experiments ran on a single physical machine over virtual networks, and this is the paper's main limitation. Every node shared one kernel, clock, CPU and storage subsystem. The shared clock is what made cross-node timestamp subtraction valid, and it is exactly what a real deployment lacks. Virtual segments are also far simpler than physical routers, BGP reconvergence, NAT, DNS failure, variable MTUs and wireless links, and attaching a bridge to two of them is trivial where a real one needs separate links, routing policy, credentials, power and an operator. The route-cut experiment therefore demonstrates a correct software response to a change in routing state, not geographic latency, independent failure domains, or real provider behaviour.

Access and subscriber isolation. The system assumes that nodes on the same network can communicate with each other or reach a shared bridge. However, some Wi-Fi networks and ISPs block direct communication between customers while still allowing them to access the provider's gateway. If this restriction exists on every available path, nodes cannot connect to each other or reach the bridge. In that situation, federation completely stops, and only local services continue to work. The current prototype has no alternative non-IP method for communication in this situation.

Network adversary. A network operator can do multiple things. They range from find stable endpoints, block IP addresses or SNI values, identify the RPC traffic, track communication timing, isolate a node to pressuring a bridge operator. Source-address binding only chooses which network connection to use; it does not hide the traffic. There is also one important failure case: **protocol whitelisting**. In this situation, normal domestic Internet routing still works, but

deep packet inspection allows only approved protocols. Since the federation uses a distinctive, non-standard protocol on a specific port, it could be blocked immediately, regardless of how strong our signatures are. Possible solutions include running federation over normal HTTPS, changing endpoints regularly, or using pluggable transports. However, none of these solutions has been implemented or tested in this work.

System and application limits. Signatures prove who published something, but they do not prove that the content is truthful or safe. A legitimate key can still be used to publish harmful content or generate a large amount of traffic. Since the admission cost is handled by the origin, an origin operator can generate as many free proofs as needed for its own users. This means that protection against abuse depends largely on the policies of the weakest origin in the peer group. The per-peer rate limits provide the main layer of protection in this case. Other problems remain open, such as stolen keys, exposed metadata, large-scale moderation, recovering from revoked keys, running out of storage, and coordinated fake identities (Sybil attacks). Transport timeouts only limit how long the system waits; they do not actually stop work that has already started. The system also specifies consistency proofs for growing trees, but these have not yet been implemented. Most importantly, a receipt only proves what the system **accepted and acknowledged**. If an operator simply refuses to accept something, no receipt is created. So, the simplest form of censorship which is silence or refusal to admit content, is impossible for this system to prove.

Evaluation limits. The route-cut figures come from single runs of one configuration, and the encoding sample is 80 activities of a single type from two deployments. We ran no multi-host, wide-area, multi-day or real-shutdown trials, and did not preserve host load, repeated-run distributions or confidence intervals. Resource figures were taken on x86 hardware and say nothing about the single-board computers a community deployment would use. Comparisons with proximity-based and voice-channel systems concern different workloads and are not a ranking.

Ethics. All tests used fake content on isolated private networks, so no real users or sensitive data were involved. However, a real deployment could put bridge operators and users at legal or physical risk because a visible relay can be identified by the network hosting it. Therefore, the system should not be deployed in a hostile environment without first assessing local threats, getting informed consent, minimizing collected data, and reviewing the deployment with the affected community.

VII. Conclusion

Server federation can survive a route cut while domestic IP paths remain. Canonical signed objects, one admission path for local and federated traffic, caller-initiated recovery and an explicitly trusted validating relay carried a dependent post across a severed exchange in 2.1 s, within a factor of two of the intact path. Cross-language byte agreement and a hop-scaling chain whose per-hop cost tracks a queueing model to within 89 ms support the mechanism from two further directions. The compact representation is 4.4–6.9× smaller than gzipped ActivityPub and forfeits interoperability, extensibility and mature tooling to get there. The most urgent work is multi-host and wide-area evaluation, consistency proofs for growing logs, cancellation-aware transport, floods of validly signed traffic, redundant independent bridges, transport camouflage against protocol whitelisting, and a non-IP fallback for subscriber isolation and total blackouts.

References

[1] Z. S. Bischof, K. Pitcher, E. Carisimo, A. Meng, R. B. Nunes, R. Padmanabhan, M. E. Roberts, A. C. Snoeren, and A. Dainotti, "Destination unreachable: Characterizing internet outages and shutdowns," in *Proceedings of the ACM SIGCOMM 2023 Conference*, 2023.

[2] T. I. Raso, S. Afrin, M. A. Chowdhury, M. Xynou, and E. Yachmeneva, "The longest silence: Internet shutdowns during Bangladesh's 2024 uprising," Digitally Right and Open Observatory of Network Interference, 2025, https://ooni.org/post/2025-bangladesh-report/.

[3] G. Baltra, T. Saluja, Y. Pradkin, and J. Heidemann, "Reasoning about internet connectivity," 2024.

[4] C. Bocovich, A. Breault, D. Fifield, Serene, and X. Wang, "Snowflake, a censorship circumvention system using temporary WebRTC proxies," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.

[5] S. Kamali and D. Barradas, "Anix: Anonymous blackout-resistant microblogging with message endorsing," in *2025 IEEE Symposium on Security and Privacy*, 2025, pp. 1381–1399.

[6] A. Lerner, G. Fanti, Y. Ben-David, J. Garcia, P. Schmitt, and B. Raghavan, "Rangzen: Anonymously getting the word out in a blackout," 2016.

[7] A. Pradeep, H. Javaid, R. Williams, A. Rault, D. Choffnes, S. Le Blond, and B. Ford, "Moby: A blackout-resistant anonymity network for mobile devices," *Proceedings on Privacy Enhancing Technologies*, vol. 2022, no. 3, pp. 247–267, 2022.

[8] C. L. Webber, J. Tallon, E. Shepherd, A. Guy, and E. Prodromou, "ActivityPub," W3C Recommendation, 2018, https://www.w3.org/TR/activitypub/.

[9] Bluesky Social, "AT Protocol repository specification," 2026, https://atproto.com/specs/repository.

[10] fiatjaf, "NIP-01: Basic protocol flow description," 2023, https://github.com/nostr-protocol/nips/blob/master/01.md.

[11] B. Laurie, E. Messeri, and R. Stradling, "Certificate transparency version 2.0," Internet Engineering Task Force, Tech. Rep. RFC 9162, 2021.

[12] A. Haeberlen, P. Kouznetsov, and P. Druschel, "PeerReview: Practical accountability for distributed systems," in *Proceedings of the 21st ACM Symposium on Operating Systems Principles*, 2007, pp. 175–188.

[13] E. Syta, I. Tamas, D. Visher, D. I. Wolinsky, P. Jovanovic, L. Gasser, N. Gailly, I. Khoffi, and B. Ford, "Keeping authorities honest or bust with decentralized witness cosigning," in *2016 IEEE Symposium on Security and Privacy*, 2016, pp. 526–545.

[14] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.

[15] Google, "Protobuf serialization is not canonical," 2024, https://protobuf.dev/programming-guides/serialization-not-canonical/.